Department of Computer Science and Engineering

CSA0607 – Design and Analysis of Algorithms

CO1 – Assessment Tool 1

IT Application-Based Problem Solving

Scalability Audit of an E-Commerce Platform (ShopSphere)

Detail	Information
Student Name	Shahid
Student Reg. No.	192424311
Course Outcome Assessed	CO1 – Determine algorithm efficiency using asymptotic notations, the Master Theorem and the substitution method
Duration / Mode	3 Days / Individual
Programming Language	Python 3 (run in VS Code)

1. About This Report

ShopSphere is an online shop. Its catalogue has **100,000 products** today and will grow to **1,000,000 products** within a year. As the algorithm engineer, I checked five back-end modules (M1 to M5) to find out which ones will become too slow, and which must be redesigned.

Rules used in every calculation

- The servers do **10^8^ basic operations per second** (100 million per second).
- The nightly job must finish in **10 minutes = 600 seconds = 6 × 10^10^ operations**.
- Time in seconds = number of basic operations ÷ 10^8.
- n always means the number of products (or records) the module works on.

How the work was done

- First I worked out each answer by hand using simple maths (shown step by step).
- Then I wrote small Python programs that count the real operations and check that the counts match my formulas.
- Every part below shows: the maths, the Python code, the program output, and what the result means for ShopSphere.

2. Part (a) – Understand the Problem

Module	Input size n	Basic operation	Where it happens	How it grows
M1	Number of products	Comparing the title and seller of two products	The IF line inside the inner FOR loop	Like n^2. Double n → 4 times the comparisons
M2	Upper limit of the request window	Audit(Request[j])	The inner FOR loop	Like n (about 2n). Double n → about double the work
M3 – S1	Products in the unsorted catalogue	Comparing the key with one product	Each step of the scan	Like n. Double n → double the comparisons
M3 – S2	Entries in the sorted index	Comparing the key with the middle entry	Each halving step	Like log n. Double n → only 1 more comparison
M4	Size of the customer–product data	Merging partial results (the "+ n" part)	After the recursive calls return	Current: like n^2. Redesign: like n log n
M5	Number of sales records	Comparing revenues while sorting	Every level of the recursion	Like n $\log^2$ n
Report generator	Number of records in the report	One step of the $3n^2$ main work	The biggest term of f(n)	Like n^2. Double n → 4 times the work

Why these choices

- **M1:** two loops, one inside the other. The IF test is the deepest, most repeated step.
- **M2:** the Audit call is the only real work. Counting Audit calls tells the whole cost.
- **S1 and S2:** a search is only a series of key comparisons, so the comparison count is the cost.
- **M4 and M5:** a divide-and-conquer algorithm splits the problem, solves the parts, then combines. The combine step (merge or sort) is the work done at every level.
- **Report generator:** f(n) is given. The n^2 term grows fastest, so it controls the cost.

3. Part (b) – Analyse the Iterative Modules

M1 – Duplicate-Listing Detector

Basic operation: one comparison of Product[i] with Product[j] (the IF line).

Step 1 – count the inner loop. For one fixed i, j goes from i + 1 to n − 1:

> Number of j values = (n − 1) − (i + 1) + 1 = n − 1 − i

Step 2 – add up over all i (i goes from 0 to n − 2):

> Total = (n − 1) + (n − 2) + (n − 3) + … + 2 + 1
> This is the sum of the first (n − 1) numbers: 1 + 2 + … + m = m(m + 1)/2, with m = n − 1
> Total = (n − 1)·n / 2 = **n(n − 1)/2** = (n² − n)/2

> **Exact comparisons = n(n − 1)/2. Time complexity = Θ(n²). Space complexity = Θ(1).**

- **Time:** the loops never stop early, so best, average and worst case are all the same: Θ(n²).
- **Space:** only the variables i, j and n are used. No extra list is created, so extra space is Θ(1).
- **Meaning:** n = 100,000 gives 4,999,950,000 comparisons. n = 1,000,000 gives 499,999,500,000. Ten times more products means about **100 times** more work.

M2 – API Rate Limiter

Basic operation: one Audit(Request[j]) call.

Step 1 – what values does i take? i starts at 1 and doubles while i < n:

> i = 1, 2, 4, 8, …, 2^m (the last one is the largest power of 2 below n)

Step 2 – the inner loop runs i times for each value of i, so the Audit count is:

> Total = 1 + 2 + 4 + 8 + … + 2^m
> Using the geometric series 1 + 2 + … + 2^m = 2^{m+1} − 1
> Total = 2^k − 1, where k = ceil($\log_2$ n) is the number of rounds

Step 3 – find the tight bound. For any n, n ≤ 2^k < 2n. So:

> $n - 1 \leq \text{Total} < 2n - 1$
> Total is between n and 2n, so Total = Θ(n)

> **Exact Audit operations = $2^k - 1$ with k = ceil($\log_2 n$). Tight bound = Θ(n). Space = Θ(1).**

Is the team member right?

No, the claim Θ(n log n) is incorrect. The team member assumed the inner loop runs n times in each round. It does not. In round one it runs 1 time, then 2, then 4, and so on. Only the last rounds are big, and everything before them adds up to about the same size as the last round ($1 + 2 + \ldots + n/4 \approx n/2$). So the total is only about 2n.

O(n log n) is still a true upper bound, but it is **not tight**. The tight bound is **Θ(n)**.

Verification for n = 1,024

> Values of i below 1024: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (10 rounds)
> Total = $1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1023 = 2^{10} - 1$
> The n log n claim would give $1024 \times 10 = 10{,}240$, which is about 10 times too large

Meaning: at n = 1,000,000 M2 does only 1,048,575 audits, which takes about 0.01 seconds. M2 is safe.

Source code

part_b_m1_m2_counts.py

```python
import math


def m1_duplicate_detector(products):
    n = len(products)
    comparisons = 0
    duplicates = []
    for i in range(0, n - 1):
        for j in range(i + 1, n):
            comparisons += 1
            if products[i] == products[j]:
                duplicates.append(j)
    return comparisons, duplicates


def m2_rate_limiter(n):
    audits = 0
    rounds = 0
    i = 1
    while i < n:
        for j in range(0, i):
            audits += 1
        i = i * 2
        rounds += 1
    return audits, rounds


print("M1 - Duplicate Listing Detector")
sample = [("Pen", "A"), ("Book", "B"), ("Pen", "A"),
          ("Bag", "C"), ("Book", "B"), ("Cup", "D")]
```

Figure 1: part_b_m1_m2_counts.py (1 of 2)

part_b_m1_m2_counts.py

```
31  count, dups = m1_duplicate_detector(sample)
32  print("Sample products :", len(sample))
33  print("Comparisons     :", count, "(formula n(n-1)/2 =", 6 * 5 // 2, ")")
34  print("Duplicates found at positions:", dups)
35  print()
36  print("  n    counted   n(n-1)/2")
37  for n in [5, 10, 100, 1000]:
38      products = [("P" + str(k), "S") for k in range(n)]
39      counted, _ = m1_duplicate_detector(products)
40      print(f"{n:>5} {counted:>9} {n * (n - 1) // 2:>10}")
41
42  print()
43  print("M2 - API Rate Limiter")
44  print("  n    counted   2^ceil(log2 n)-1   n*log2(n)")
45  for n in [8, 100, 1000, 1024, 5000]:
46      counted, rounds = m2_rate_limiter(n)
47      formula = 2 ** math.ceil(math.log2(n)) - 1
48      print(f"{n:>5} {counted:>9} {formula:>17} {n * math.log2(n):>12.0f}")
49
50  print()
51  print("Check for n = 1024")
52  total = 0
53  i = 1
54  while i < 1024:
55      print("  i =", i, "-> inner loop runs", i, "times")
56      total += i
57      i = i * 2
58  print("Total Audit operations =", total)
59  print("Claimed n log n        =", 1024 * 10)
```

Figure 1: part_b_m1_m2_counts.py (2 of 2)

Output

```
TERMINAL
> python part_b_m1_m2_counts.py
M1 - Duplicate Listing Detector
Sample products : 6
Comparisons     : 15 (formula n(n-1)/2 = 15 )
Duplicates found at positions: [2, 4]

  n    counted   n(n-1)/2
    5        10         10
   10        45         45
  100      4950       4950
 1000    499500     499500

M2 - API Rate Limiter
  n    counted   2^ceil(log2 n)-1   n*log2(n)
    8         7                 7          24
  100       127               127         664
 1000      1023              1023        9966
 1024      1023              1023       10240
 5000      8191              8191       61439

Check for n = 1024
  i = 1 -> inner loop runs 1 times
  i = 2 -> inner loop runs 2 times
  i = 4 -> inner loop runs 4 times
  i = 8 -> inner loop runs 8 times
  i = 16 -> inner loop runs 16 times
  i = 32 -> inner loop runs 32 times
  i = 64 -> inner loop runs 64 times
  i = 128 -> inner loop runs 128 times
  i = 256 -> inner loop runs 256 times
  i = 512 -> inner loop runs 512 times
Total Audit operations = 1023
Claimed n log n        = 10240
```

Figure 2: Output of part_b_m1_m2_counts.py

The output shows the counted values equal the formulas $n(n-1)/2$ and $2^k - 1$ for every n tested, and that M2 gives 1023 for n = 1024.

4. Part (c) – Solve a Recurrence by Substitution

Binary search (S2) compares the key with the middle entry, then throws away half. Its comparison count is:

> $T(n) = T(n/2) + 1, \quad T(1) = 1$

Backward substitution – step by step

> Step 1: $T(n) = T(n/2) + 1$
> Step 2: Put $T(n/2) = T(n/4) + 1 \rightarrow T(n) = T(n/4) + 1 + 1 = T(n/4) + 2$
> Step 3: Put $T(n/4) = T(n/8) + 1 \rightarrow T(n) = T(n/8) + 1 + 1 + 1 = T(n/8) + 3$
> Step 4: Put $T(n/8) = T(n/16) + 1 \rightarrow T(n) = T(n/16) + 4$

Spot the pattern. After k steps:

> $T(n) = T(n / 2^k) + k$

Stop at the base case. The recursion stops when $n / 2^k = 1$:

> $n / 2^k = 1 \rightarrow 2^k = n \rightarrow k = \log_2 n$
> $T(n) = T(1) + \log_2 n = 1 + \log_2 n$

> **Exact closed form: $T(n) = \log_2 n + 1$ (n a power of 2). For any n: floor($\log_2 n$) + 1.**

Quick check, n = 8: $T(8) = T(4) + 1 = T(2) + 2 = T(1) + 3 = 1 + 3 = 4$, and $\log_2 8 + 1 = 3 + 1 = 4$. ✓

Time and space complexity

Version	Time	Space	Why
Iterative	Θ(log n)	Θ(1)	Only low, high and mid are stored. The loop runs about $\log_2 n$ times.
Recursive	Θ(log n)	Θ(log n)	Each call waits on the call stack. The stack gets about $\log_2 n$ calls deep.

Meaning: for 1,000,000 products binary search needs only **20 comparisons**. Doubling the catalogue adds just one more comparison. The program confirms 20 comparisons and a recursion depth of 20.

Source code

part_c_binary_search.py

```
1  import math
2
3
4  def binary_search_iterative(arr, key):
5      low, high = 0, len(arr) - 1
6      comparisons = 0
7      while low <= high:
8          mid = (low + high) // 2
9          comparisons += 1
10         if arr[mid] == key:
11             return mid, comparisons
12         elif key < arr[mid]:
13             high = mid - 1
14         else:
15             low = mid + 1
16     return -1, comparisons
17
18
19 def binary_search_recursive(arr, key, low, high, depth=1):
20     if low > high:
21         return -1, 0, depth
22     mid = (low + high) // 2
23     if arr[mid] == key:
24         return mid, 1, depth
25     if key < arr[mid]:
26         idx, c, d = binary_search_recursive(arr, key, low, mid - 1, depth + 1)
27     else:
28         idx, c, d = binary_search_recursive(arr, key, mid + 1, high, depth + 1)
29     return idx, c + 1, d
30
31
32 def T(n):
33     if n == 1:
```

Figure 3: part_c_binary_search.py (1 of 2)

Figure 3: part_c_binary_search.py (2 of 2)

part_c_binary_search.py

```
34          return 1
35      return T(n // 2) + 1
36
37
38  print("Backward substitution for n = 1024")
39  n = 1024
40  k = 0
41  while n >= 1:
42      print(f"  k = {k:>2}:  T(1024) = T({n}) + {k}")
43      n = n // 2
44      k += 1
45  print("  T(1) = 1, so T(1024) = 1 + 10 =", T(1024))
46
47  print()
48  print("  n       worst case   log2(n)+1")
49  for p in range(1, 15):
50      n = 2 ** p
51      arr = [2 * x for x in range(n)]
52      worst = 0
53      for key in range(-1, 2 * n + 2):
54          found, c = binary_search_iterative(arr, key)
55          worst = max(worst, c)
56      print(f"{n:>6} {worst:>12} {int(math.log2(n)) + 1:>12}")
57
58  print()
59  n = 1000000
60  arr = [2 * x for x in range(n)]
61  found, c = binary_search_iterative(arr, 2 * n - 2)
62  print("n = 1,000,000, iterative, search for last item:", c, "comparisons")
63  found, c, depth = binary_search_recursive(arr, 2 * n - 2, 0, n - 1)
64  print("n = 1,000,000, recursive, comparisons:", c, " stack depth:", depth)
65  print("floor(log2 n) + 1 =", int(math.log2(n)) + 1)
```

Output

```
TERMINAL
> python part_c_binary_search.py
Backward substitution for n = 1024
  k =  0:  T(1024) = T(1024) + 0
  k =  1:  T(1024) = T(512) + 1
  k =  2:  T(1024) = T(256) + 2
  k =  3:  T(1024) = T(128) + 3
  k =  4:  T(1024) = T(64) + 4
  k =  5:  T(1024) = T(32) + 5
  k =  6:  T(1024) = T(16) + 6
  k =  7:  T(1024) = T(8) + 7
  k =  8:  T(1024) = T(4) + 8
  k =  9:  T(1024) = T(2) + 9
  k = 10:  T(1024) = T(1) + 10
  T(1) = 1, so T(1024) = 1 + 10 = 11

  n       worst case   log2(n)+1
     2            2            2
     4            3            3
     8            4            4
    16            5            5
    32            6            6
    64            7            7
   128            8            8
   256            9            9
   512           10           10
  1024           11           11
  2048           12           12
  4096           13           13
  8192           14           14
 16384           15           15

n = 1,000,000, iterative, search for last item: 20 comparisons
n = 1,000,000, recursive, comparisons: 20  stack depth: 20
floor(log2 n) + 1 = 20
```

Figure 4: Output of part_c_binary_search.py

The table in the output tries every possible key for n = 2 up to 16,384. The worst case always equals $\log_2 n + 1$, as the formula says.

5. Part (d) – Apply the Master Theorem

For $T(n) = a \cdot T(n/b) + f(n)$, compare $f(n)$ with n^p, where $p = \log_b a$:

- **Case 1:** $f(n)$ is smaller than n^p (by a power of n) $\rightarrow$ $T(n) = \Theta(n^p)$
- **Case 2:** $f(n)$ is the same size as n^p $\rightarrow$ $T(n) = \Theta(n^p \log n)$
- **Case 3:** $f(n)$ is bigger than n^p (by a power of n) $\rightarrow$ $T(n) = \Theta(f(n))$

M4 – Recommendation Engine

	Current design	Proposed redesign
Recurrence	$T(n) = 4T(n/2) + n$	$T(n) = 2T(n/2) + n$
a	4	2
b	2	2
$f(n)$	n	n
n^p ($p = \log_b a$)	$p = \log_2 4 = 2$, so $n^p = n^2$	$p = \log_2 2 = 1$, so $n^p = n$
Compare	n is smaller than n^2 ($n = O(n^{2-\varepsilon})$, with $\varepsilon = 1$)	n is the same as n
Case used	Case 1	Case 2
Result	$T(n) = \Theta(n^2)$	$T(n) = \Theta(n \log n)$

What happens when the data doubles?

> Current: $T(2n) / T(n) \approx (2n)^2 / n^2 = \mathbf{4}$ $\rightarrow$ double the data, 4 times the time
> Redesign: $T(2n) / T(n) \approx 2n \cdot \log(2n) / n \cdot \log n = 2 \cdot (1 + 1/\log_2 n) \approx \mathbf{2.1}$ for n near a million

Meaning: removing the replicated partitions changes M4 from $\Theta(n^2)$ to $\Theta(n \log n)$. At n = 1,000,000 that is 2×10^{12} operations down to 2×10^7, which is about **95,000 times less work**.

M5 – Sales Analytics Engine: $T(n) = 2T(n/2) + n \log n$

> $a = 2$, $b = 2$, $f(n) = n \log n$, $p = \log_2 2 = 1$, so $n^p = n$

Is any standard case a fit?

- **Case 1** needs $f(n)$ smaller than n by a power of n. But $n \log n$ is bigger than n. ✗
- **Case 2** needs $f(n) = \Theta(n)$. But $n \log n$ has an extra $\log n$ factor. ✗
- **Case 3** needs $f(n)$ bigger than n by a power of n (like $n^{1+\varepsilon}$). But $\log n$ grows slower than any power of n. ✗

No standard case applies. I solve it in two ways.

Method 1: Extended Master Theorem

> If $f(n) = \Theta(n^p \cdot \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^p \cdot \log^{k+1} n)$
> Here $p = 1$ and $f(n) = n \cdot \log^1 n$, so $k = 1$
> $T(n) = \Theta(n \cdot \log^2 n)$

Method 2: Recursion tree

Level 0: 1 problem of size n → cost n log n
Level 1: 2 problems of size n/2 → cost 2 · (n/2) log(n/2) = n(log n − 1)
Level 2: 4 problems of size n/4 → cost n(log n − 2)
Level i: 2^i problems of size $n/2^i$ → cost n(log n − i)
Number of levels: $L = \log_2 n$ (levels 0 to L−1), plus n leaves of cost 1 each

$T(n) = n \cdot [L + (L-1) + \ldots + 2 + 1] + n$
$= n \cdot L(L+1)/2 + n$
$\approx \frac{1}{2} \cdot n \log^2 n \rightarrow \Theta(n \log^2 n)$

M4 current = Θ(n²). M4 redesign = Θ(n log n). M5 = Θ(n log² n) (both methods agree).

Meaning: M5 grows only a little faster than n log n. At n = 1,000,000 it needs about 2.1 × 10^8 operations (2.1 seconds), which is fine.

Source code

part_d_master_theorem.py

```python
import math


def master_theorem(a, b, d):
    p = math.log(a, b)
    if round(d, 9) < round(p, 9):
        return "Case 1: Theta(n^" + str(round(p)) + ")"
    if round(d, 9) == round(p, 9):
        if round(p) == 1:
            return "Case 2: Theta(n log n)"
        return "Case 2: Theta(n^" + str(round(p)) + " log n)"
    return "Case 3: Theta(n^" + str(d) + ")"


def solve(a, f, kmax):
    values = [1]
    for k in range(1, kmax + 1):
        values.append(a * values[k - 1] + f(2 ** k, k))
    return values


print("M4 current   T(n) = 4T(n/2) + n : a=4 b=2 n^(log2 4)=n^2 f(n)=n")
print("  ", master_theorem(4, 2, 1))
print("M4 redesign  T(n) = 2T(n/2) + n : a=2 b=2 n^(log2 2)=n   f(n)=n")
print("  ", master_theorem(2, 2, 1))
print("M5  T(n) = 2T(n/2) + n log n : f(n)=n log n, n^(log2 2)=n")
print("   Not Case 1, 2 or 3 -> use recursion tree: Theta(n log^2 n)")
```

Figure 5: part_d_master_theorem.py (1 of 2)

part_d_master_theorem.py

```
old = solve(4, lambda n, k: n, 20)
new = solve(2, lambda n, k: n, 20)
m5 = solve(2, lambda n, k: n * k, 20)

print()
print("  k        n     T_old(n)    2n^2-n    T_new(n)   n log n + n")
for k in [3, 10, 20]:
    n = 2 ** k
    print(f"{k:>3} {n:>8} {old[k]:>12} {2 * n * n - n:>9} "
          f"{new[k]:>11} {n * k + n:>11}")

print()
print("Doubling the data (n = 2^19 -> 2^20)")
print("  current  T(2n)/T(n) =", round(old[20] / old[19], 3))
print("  redesign T(2n)/T(n) =", round(new[20] / new[19], 3))
print("  M5       T(2n)/T(n) =", round(m5[20] / m5[19], 3))

print()
print("M5: T(n) / (n log^2 n) settles near a constant (0.5)")
for k in [5, 10, 15, 20]:
    n = 2 ** k
    print(f"  n = 2^{k:<2}  T/(n log n) = {m5[k] / (n * k):5.2f}"
          f"   T/(n log^2 n) = {m5[k] / (n * k * k):.3f}")
exact = all(m5[k] == 2 ** k * k * (k + 1) // 2 + 2 ** k for k in range(21))
print("Exact form T(n) = n*k(k+1)/2 + n matches for every k:", exact)

```

Figure 5: part_d_master_theorem.py (2 of 2)

Output

```
TERMINAL

> python part_d_master_theorem.py
M4 current   T(n) = 4T(n/2) + n : a=4 b=2 n^(log2 4)=n^2 f(n)=n
   Case 1: Theta(n^2)
M4 redesign  T(n) = 2T(n/2) + n : a=2 b=2 n^(log2 2)=n   f(n)=n
   Case 2: Theta(n log n)
M5  T(n) = 2T(n/2) + n log n : f(n)=n log n, n^(log2 2)=n
   Not Case 1, 2 or 3 -> use recursion tree: Theta(n log^2 n)

  k        n     T_old(n)    2n^2-n    T_new(n)   n log n + n
  3        8          120       120          32          32
 10     1024      2096128   2096128       11264       11264
 20  1048576 2199022206976 2199022206976    22020096    22020096

Doubling the data (n = 2^19 -> 2^20)
  current  T(2n)/T(n) = 4.0
  redesign T(2n)/T(n) = 2.1
  M5       T(2n)/T(n) = 2.209

M5: T(n) / (n log^2 n) settles near a constant (0.5)
  n = 2^5   T/(n log n) =  3.20   T/(n log^2 n) = 0.640
  n = 2^10  T/(n log n) =  5.60   T/(n log^2 n) = 0.560
  n = 2^15  T/(n log n) =  8.07   T/(n log^2 n) = 0.538
  n = 2^20  T/(n log n) = 10.55   T/(n log^2 n) = 0.527
Exact form T(n) = n*k(k+1)/2 + n matches for every k: True
```

Figure 6: Output of part_d_master_theorem.py

The program builds T(n) from the recurrences up to n = 2^{20}. The exact formulas $2n^2 - n$ and n log n + n match for M4, and for M5 the ratio $T(n)/(n \log^2 n)$ settles near 0.5.

6. Part (e) – Best, Average and Worst Cases

Case	S1 – condition	S1 comparisons	S2 – condition	S2 comparisons
Best	Product is the first one in the list	1	Product is exactly the middle entry	1
Worst	Product is the last one, or is not in the catalogue	n	Product is at the deepest level, or is missing	floor($\log_2 n$) + 1 = 20 for n = 10^6
Average	Each position equally likely (product found)	(n + 1)/2	Each position equally likely (product found)	about $\log_2 n - 1$ = 18.95 for n = 10^6

S1 – average comparisons with probability p

Let p be the chance that the product exists. If it exists, every position 1…n is equally likely. If it does not exist (chance 1 − p), all n products are checked.

> If found: average = (1 + 2 + … + n) / n = (n + 1) / 2
> If not found: comparisons = n
> A(n) = p · (n + 1)/2 + (1 − p) · n
> = p(n + 1)/2 + n − pn
> = **n(1 − p/2) + p/2**

Evaluate for n = 1,000,000 and p = 0.5:

> A = 0.5 × (1,000,001 / 2) + 0.5 × 1,000,000
> = 0.5 × 500,000.5 + 500,000
> = 250,000.25 + 500,000
> = **750,000.25 comparisons**

> **S1 average = 750,000.25 comparisons (about 0.0075 s). S2 needs at most 20.**

Meaning: sequential search needs about 37,500 times more comparisons than binary search on a million products. ShopSphere should keep a sorted index and use S2 for product search.

Source code

part_e_search_cases.py

```python
import math
from functools import lru_cache


def sequential_search(arr, key):
    comparisons = 0
    for x in arr:
        comparisons += 1
        if x == key:
            return comparisons
    return comparisons


def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        if arr[mid] == key:
            return comparisons
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return comparisons


@lru_cache(maxsize=None)
def total_comparisons(n):
    if n == 0:
        return 0
    left = (n - 1) // 2
    return n + total_comparisons(left) + total_comparisons(n - 1 - left)


```

Figure 7: part_e_search_cases.py (1 of 2)

part_e_search_cases.py

```
print("S1 - Sequential search, n = 10")
arr = list(range(10))
print("  best  (first item)        :", sequential_search(arr, 0))
print("  worst (last item / absent):", sequential_search(arr, 9),
      "/", sequential_search(arr, 99))
found_avg = sum(sequential_search(arr, k) for k in arr) / 10
print("  average when found        :", found_avg, "= (n+1)/2")
p = 0.5
avg = p * found_avg + (1 - p) * 10
print("  average with p = 0.5      :", avg, "= n(1 - p/2) + p/2")

print()
n = 1000000
p = 0.5
print("S1, n = 1,000,000 and p = 0.5")
print("  found   : (n+1)/2 =", (n + 1) / 2)
print("  absent  : n       =", n)
print("  average : 0.5 x 500000.5 + 0.5 x 1000000 =",
      p * (n + 1) / 2 + (1 - p) * n)

print()
print("S2 - Binary search, n = 15 (every item tried)")
arr = list(range(15))
counts = [binary_search(arr, k) for k in arr]
print("  best :", min(counts), " worst:", max(counts),
      " average:", round(sum(counts) / 15, 3))

print()
print("S2, n = 1,000,000")
print("  best    : 1 comparison (key is the middle item)")
print("  worst   :", int(math.log2(n)) + 1, "comparisons = floor(log2 n) + 1")
print("  average :", round(total_comparisons(n) / n, 3),
      "comparisons (about log2 n - 1)")
check = list(range(1000))
real = sum(binary_search(check, k) for k in check)
print("  formula check for n = 1000:", real == total_comparisons(1000))
```

Figure 7: part_e_search_cases.py (2 of 2)

Output

```
TERMINAL
> python part_e_search_cases.py
S1 - Sequential search, n = 10
  best  (first item)        : 1
  worst (last item / absent): 10 / 10
  average when found        : 5.5 = (n+1)/2
  average with p = 0.5      : 7.75 = n(1 - p/2) + p/2

S1, n = 1,000,000 and p = 0.5
  found   : (n+1)/2 = 500000.5
  absent  : n       = 1000000
  average : 0.5 x 500000.5 + 0.5 x 1000000 = 750000.25

S2 - Binary search, n = 15 (every item tried)
  best : 1  worst: 4  average: 3.267

S2, n = 1,000,000
  best    : 1 comparison (key is the middle item)
  worst   : 20 comparisons = floor(log2 n) + 1
  average : 18.951 comparisons (about log2 n - 1)
  formula check for n = 1000: True
```

Figure 8: Output of part_e_search_cases.py

7. Part (f) – Prove an Asymptotic Bound

Definition. $f(n) = \Theta(n^2)$ means there are positive constants c_1, c_2 and n_0 such that:

> $c_1 \cdot n^2 \le f(n) \le c_2 \cdot n^2$ for every $n \ge n_0$

Here $f(n) = 3n^2 + 10\, n \log_2 n + 50$.

Lower bound

> For $n \ge 1$, the terms $10\, n \log_2 n$ and 50 are not negative.
> So $f(n) \ge 3n^2$ → take **$c_1 = 3$**

Upper bound

> For $n \ge 1$ we know $\log_2 n \le n$, so $10\, n \log_2 n \le 10\, n \cdot n = 10n^2$
> For $n \ge 1$ we know $n^2 \ge 1$, so $50 \le 50n^2$
> $f(n) \le 3n^2 + 10n^2 + 50n^2 = 63n^2$ → take **$c_2 = 63$**

$3n^2 \le f(n) \le 63n^2$ for all $n \ge 1$. So $c_1 = 3$, $c_2 = 63$, $n_0 = 1$, and $f(n) = \Theta(n^2)$.

Show that f(n) is not O(n)

Use proof by contradiction. Suppose $f(n) = O(n)$. Then some constant c and some n_0 exist with $f(n) \le c \cdot n$ for all $n \ge n_0$.

> We already know $f(n) \ge 3n^2$.
> So $3n^2 \le f(n) \le c \cdot n$ → $3n^2 \le c \cdot n$ → $n \le c/3$
> This would have to be true for every $n \ge n_0$, including n larger than $c/3$.
> That is impossible. So $f(n)$ is **not** $O(n)$.

Example: for $c = 1000$ the inequality $f(n) \le 1000n$ fails as soon as $n = 306$. For $c = 100{,}000$ it fails at $n = 33{,}284$. Bigger c only delays the failure.

Meaning: the report generator cost is really quadratic. At $n = 1{,}000{,}000$ it needs about 3×10^{12} operations, which is far beyond the 10-minute window.

Source code

part_f_asymptotic_proof.py

```python
import math


def f(n):
    return 3 * n * n + 10 * n * math.log2(n) + 50


c1 = 3
c2 = 63
n0 = 1

print("Claim: c1 * n^2 <= f(n) <= c2 * n^2 for all n >= n0")
print("c1 =", c1, " c2 =", c2, " n0 =", n0)

ok = True
smallest = 1000
largest = 0
for n in range(n0, 1000001):
    ratio = f(n) / (n * n)
    smallest = min(smallest, ratio)
```

Figure 9: part_f_asymptotic_proof.py (1 of 2)

Output

TERMINAL

```
> python part_f_asymptotic_proof.py
Claim: c1 * n^2 <= f(n) <= c2 * n^2 for all n >= n0
c1 = 3  c2 = 63  n0 = 1
Checked every n from 1 to 1,000,000 -> bound holds
Smallest f(n)/n^2 = 3.0002
Largest  f(n)/n^2 = 53.0

  n          f(n)         f(n)/n^2     f(n)/n
     10           682       6.8219        68.2
    100         36694       3.6694       366.9
   1000       3099708       3.0997      3099.7
 100000   30016609690       3.0017    300166.1
1000000 3000199315736       3.0002   3000199.3

f(n) is not O(n): f(n)/n keeps growing, so no constant c works
  for c =    100, f(n) > c*n already at n = 19
  for c =   1000, f(n) > c*n already at n = 306
  for c = 100000, f(n) > c*n already at n = 33284
```

Figure 10: Output of part_f_asymptotic_proof.py

The program checked every n from 1 to 1,000,000. The bound $3n^2 \le f(n) \le 63n^2$ held every time, and $f(n)/n^2$ moves towards 3.

8. Part (g) – Scalability Decision

Operation counts come from the exact formulas found earlier. Time = operations ÷ 10^8. The limit is 600 seconds (6×10^{10} operations).

Estimates for n = 1,000,000

Module	Complexity	Operations	Time
M1	Θ(n²)	499,999,500,000	83.3 minutes
M2	Θ(n)	1,048,575	0.010 seconds
S2	Θ(log n)	20	0.2 microseconds
M4 current	Θ(n²)	1,999,999,000,000	5.6 hours
M4 redesign	Θ(n log n)	20,931,569	0.209 seconds
M5	Θ(n log² n)	209,599,497	2.1 seconds
Report generator	Θ(n²)	3,000,199,315,736	8.3 hours

Today (100,000) compared with next year (1,000,000)

Module	Time at n = 100,000	Time at n = 1,000,000	Growth for 10× more products
M1	50.0 seconds	83.3 minutes	100 times
M2	0.001 seconds	0.010 seconds	8 times
S2	0.17 microseconds	0.20 microseconds	+3 comparisons
M4 current	3.3 minutes	5.6 hours	100 times
M4 redesign	0.018 seconds	0.209 seconds	about 12 times
M5	0.147 seconds	2.1 seconds	about 14 times
Report generator	5.0 minutes	8.3 hours	100 times

Important point: all modules pass today. M1, the current M4 and the report generator pass only because n is small. When ShopSphere grows ten times, their time grows a hundred times, and they fail.

Which modules must be redesigned?

Module	Problem	Recommended redesign
M1 Duplicate detector	$\Theta(n^2)$. Takes 83 minutes at $n = 10^6$.	Use a hash set (Python set) of (title, seller). Go through the products once; if the pair is already in the set it is a duplicate. This is about $\Theta(n)$ time. (Other option: sort by title and seller, then compare neighbours, $\Theta(n \log n)$.)
M4 Current design	$\Theta(n^2)$. Takes 5.6 hours.	Adopt the proposed redesign (no replicated partitions). It becomes $\Theta(n \log n)$ and takes 0.2 seconds.
Report generator	$\Theta(n^2)$. Takes 8.3 hours.	Find the nested loop behind the $3n^2$ term. Replace it with sorting or hash-based grouping so the work is about $n \log n$. (The exact fix depends on the report code, which is not given here.)

Modules that can stay as they are

- **M2:** $\Theta(n)$, 0.01 seconds. Fine.
- **S2:** 20 comparisons. Fine. Use S2 and keep a sorted index; S1 would need about 750,000 comparisons per search on average.
- **M4 redesign:** 0.2 seconds. Fine.
- **M5:** 2.1 seconds. Fine. (Optional: sort once instead of at every level to reach $\Theta(n \log n)$.)

Proof that the M1 redesign works

I ran the old nested-loop method and the hash-set method on 4,000 sample products. Both found the same 1,060 duplicates, but the hash set was about 500 times faster even at this small size, and the gap grows with n.

Source code

part_g_scalability.py

```python
import math
import random
import time

SPEED = 100000000
WINDOW = 600


def nice_time(seconds):
    if seconds < 0.001:
        return f"{seconds * 1000000:.2f} microseconds"
    if seconds < 60:
        return f"{seconds:.3f} seconds"
    if seconds < 7200:
        return f"{seconds / 60:.1f} minutes"
    return f"{seconds / 3600:.1f} hours"


def operations(n):
    L = math.log2(n)
    return [
        ("M1 duplicate detector", n * (n - 1) // 2),
        ("M2 rate limiter", 2 ** math.ceil(L) - 1),
        ("S2 binary search", int(L) + 1),
        ("M4 current design", 2 * n * n - n),
        ("M4 redesign", n * L + n),
        ("M5 analytics engine", n * L * (L + 1) / 2 + n),
```

Figure 11: part_g_scalability.py (1 of 3)

part_g_scalability.py

```
        ("Report generator f(n)", 3 * n * n + 10 * n * L + 50),
    ]


for n in [100000, 1000000]:
    print(f"n = {n:,}")
    print(f"{'Module':<24}{'Operations':>20}{'Time':>20}  Status")
    for name, ops in operations(n):
        seconds = ops / SPEED
        status = "OK" if seconds <= WINDOW else "FAILS"
        print(f"{name:<24}{ops:>20,.0f}{nice_time(seconds):>20}  {status}")
    print()


def find_duplicates_slow(products):
    found = set()
    for i in range(len(products) - 1):
        for j in range(i + 1, len(products)):
            if products[i] == products[j]:
                found.add(j)
    return found


def find_duplicates_fast(products):
    seen = set()
    found = set()
    for j in range(len(products)):
```

Figure 11: part_g_scalability.py (2 of 3)

part_g_scalability.py

```
55          if products[j] in seen:
56              found.add(j)
57          else:
58              seen.add(products[j])
59      return found
60
61
62  random.seed(1)
63  products = []
64  for _ in range(4000):
65      title = "Item" + str(random.randint(1, 2000))
66      seller = "S" + str(random.randint(1, 3))
67      products.append((title, seller))
68  start = time.time()
69  slow = find_duplicates_slow(products)
70  slow_time = time.time() - start
71  start = time.time()
72  fast = find_duplicates_fast(products)
73  fast_time = time.time() - start
74  print("M1 redesign test with 4,000 products")
75  print("  nested loops :", len(slow), "duplicates,",
76        round(slow_time, 3), "seconds")
77  print("  hash set     :", len(fast), "duplicates,",
78        round(fast_time, 5), "seconds")
79  print("  same answer  :", slow == fast)
```

Figure 11: part_g_scalability.py (3 of 3)

Output

```
TERMINAL
> python part_g_scalability.py
n = 100,000
Module                          Operations                Time  Status
M1 duplicate detector        4,999,950,000      49.999 seconds  OK
M2 rate limiter                    131,071       0.001 seconds  OK
S2 binary search                        17   0.17 microseconds  OK
M4 current design           19,999,900,000         3.3 minutes  OK
M4 redesign                      1,760,964       0.018 seconds  OK
M5 analytics engine             14,724,490       0.147 seconds  OK
Report generator f(n)       30,016,609,690         5.0 minutes  OK

n = 1,000,000
Module                          Operations                Time  Status
M1 duplicate detector       499,999,500,000        83.3 minutes  FAILS
M2 rate limiter                  1,048,575       0.010 seconds  OK
S2 binary search                        20   0.20 microseconds  OK
M4 current design        1,999,999,000,000           5.6 hours  FAILS
M4 redesign                     20,931,569       0.209 seconds  OK
M5 analytics engine            209,599,497       2.096 seconds  OK
Report generator f(n)    3,000,199,315,736           8.3 hours  FAILS

M1 redesign test with 4,000 products
  nested loops : 1060 duplicates, 0.412 seconds
  hash set     : 1060 duplicates, 0.00079 seconds
  same answer  : True
```

Figure 12: Output of part_g_scalability.py

The measured seconds for the 4,000-product test depend on the computer, so your own run may show slightly different times. The duplicate counts will be the same.

9. Quick Viva Answers

Question	Short answer
What is a basic operation?	The step repeated the most, which decides the running time (for example the comparison in the inner loop).
Why is M2 $\Theta(n)$ and not $\Theta(n \log n)$?	The inner loop runs i times, not n times, and i doubles each round. $1 + 2 + 4 + \ldots + n/2$ is about n, so the total is about 2n.
What is the formula for M1?	$n(n-1)/2$ comparisons, which is $\Theta(n^2)$.
What do O, Ω and Θ mean?	O is an upper bound, Ω is a lower bound, Θ is both (a tight bound).
State the three Master Theorem cases.	Compare $f(n)$ with n^p, where $p = \log_b a$. Smaller → $\Theta(n^p)$. Equal → $\Theta(n^p \log n)$. Bigger → $\Theta(f(n))$.
Why can't the Master Theorem solve $2T(n/2) + n \log n$?	$n \log n$ is bigger than n but not by a power of n, and it is not equal to n either. So none of the 3 cases fit; use the extended theorem or a recursion tree. Answer $\Theta(n \log^2 n)$.
Why does recursive binary search use $\Theta(\log n)$ space?	Each recursive call stays on the stack until the call below it finishes, and there are about $\log_2 n$ calls.
How did you choose c_1 and c_2 in part (f)?	$c_1 = 3$ by dropping the other positive terms. $c_2 = 63$ by using $\log n \le n$ and $50 \le 50n^2$ so everything is at most a multiple of n^2.
Why does a hash set fix M1?	Looking up an item in a hash set takes about constant time, so one pass over n products costs about n, instead of comparing all pairs.

10. Repository Structure and How to Run

```
CO1_AT1/
│
├── Question_1/
│    ├── Source_Code/
│    │    ├── part_b_m1_m2_counts.py
│    │    ├── part_c_binary_search.py
│    │    ├── part_d_master_theorem.py
│    │    ├── part_e_search_cases.py
│    │    ├── part_f_asymptotic_proof.py
│    │    ├── part_g_scalability.py
│    │    └── run_all.py
│    └── Report/
│         ├── CSA06_CO1_AT1_Report.docx
│         ├── screenshots/   (code and output
images)
│         └── outputs/(program output as text
files)
└── README.md
```

Code of Conduct

I **SHAIK SHAHID (192424311)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Date:**11-10-2026**